

Raras Technology



Mizan-Tepi University



School of Computing & Informatics

Style Definition: Default Paragraph Font

StudentPortal

Web-Based Academic Management System

Internship Project Documentation

Submitted by: Mut Gatkek

Degree: BSc in Information Technology

Supervisor: Getenet A

Department: Information Technology

Date: July 2026

Internship Company: RRT Technology

Submitted to Company: **RRT.**

Academic Year 2026/2027

Submission date: **JULY,2026 G.C**

Location: A.A, Ethiopia

RRT Company Internship Program 2026

Prepared by Mut Gatkek, BSc in Information Technology | **July,2026**

Acknowledgment

First and foremost, I would like to express my deepest gratitude to the Almighty God for giving me the strength, wisdom, and perseverance to complete this project successfully.

I am profoundly grateful to RRT Company for providing me with the opportunity to undertake my internship program and for the invaluable experience I gained during this period. The company's commitment to fostering young talent and promoting innovation has been instrumental in shaping my professional development.

I would like to extend my sincere thanks to my internship supervisor at RRT Company for their continuous guidance, mentorship, and constructive feedback throughout the project. Their expertise and patience have been a great source of inspiration.

I am also thankful to my university for the theoretical foundation and academic support that made this project possible. The knowledge gained during my studies in Information Technology provided the essential groundwork for developing the StudentPortal system.

My heartfelt appreciation goes to the team members at Raras (RRT Technology) who collaborated with me, shared their knowledge, and provided technical support whenever needed. Their professionalism and willingness to help made the development process a rewarding learning experience.

Finally, I would like to thank my family and friends for their unwavering support, encouragement, and understanding throughout this journey.

Table of Contents

Acknowledgment	i
List of Figures	vi
List of Tables	vii
List of Abbreviations and Acronyms	viii
Abstract	ix
Chapter One: Introduction	1
1.1 Background of RRT Company	1
1.2 Background of the Project	1
1.3 Statement of the Problem	2
1.4 Objectives of the Project	4
1.4.1 General Objective	4
1.4.2 Specific Objectives	4
1.5 Methodology	5
1.5.1 Data Sources	5
1.5.1.1 Primary Data Source	5
1.5.1.2 Secondary Data Source	5
1.5.2 Data Gathering Techniques	6
1.5.3 Systems Analysis and Design	6
1.5.4 Overview of the Project	7
1.5.5 Development Tools and Technology	7
1.6 Scope of the Project	8
1.7 Feasibility Study	9
1.7.1 Operational Feasibility	9

1.7.2 Technical Feasibility	10
1.7.3 Behavioral/Political Feasibility.....	10
1.7.4 Economic Feasibility	10
1.8 Limitations of the Project.....	10
1.9 Significance of the Project.....	11
1.10 Task and Schedule	12
1.11 Cost Analysis of the Project.....	13
1.12 Communication Plan.....	13
1.13 Project Team Composition	14
Chapter Two: Existing System	15
2.1 Description of the Existing System.....	15
2.2 Problems of the Existing System	15
2.2.1 Performance Issues	15
2.2.2 Security and Control Issues	16
2.2.3 Economic Issues.....	16
2.2.4 Data Storage Issues.....	16
2.2.5 Efficiency Issues	16
2.3 Actors in the Existing System.....	17
2.4 Business Rules of the System	17
2.5 Reports Generated in the Existing System.....	17
2.6 Alternative Options to Address Problems.....	18
Chapter Three: Proposed System.....	19
3.1 Overview of the Proposed System.....	19
3.2 Functional Requirements	19
3.3 Non-Functional Requirements	20

Chapter Four: System Design	22
4.1 System Design Using Object Oriented Paradigm.....	22
4.1.1 Design Goals.....	22
4.2 Use Case Diagram.....	22
4.2.1 Actor and Use Case Identification	22
4.3 Use Case Diagram.....	25
4.3.1 Use Case Descriptions	25
4.4 Sequence Diagrams.....	27
4.5 Activity Diagrams.....	29
4.6 Class Diagram.....	31
4.7 Persistent Modeling / Database Design	32
4.8 Component Diagram.....	33
4.9 Deployment Diagram.....	34
4.10 Object Diagram.....	35
Chapter Five: System Implementation.....	36
5.1 Introduction.....	36
5.2 Sample code	36
5.3 User interface	39
5.4 Testing.....	40
5.6 Training.....	40
5.7 Start-up strategy	41
5.6 Conclusion and recommendation.....	41
56.1 Conclusion	41
56.2 Recommendations.....	42
6.2 References.....	43

Appendix.....	44
Appendix A: Source Code Structure.....	44
Appendix B: API Documentation (Swagger)	45
Appendix C: Database Schema.....	46
Appendix D: Testing Plan.....	47

List of Figures

Table 1.1: Software Tools.....	7
Table 1.2: Technology Stack	8
Table 1.3: Hardware Tools	8
Table 1.4: Project Schedule	12
Table 1.5: Project Cost Analysis.....	13
Table 1.6: Communication Plan	13
Table 1.7: Project Team.....	14
Table 4.1: Actor Identification.....	23
Table 4.2: Use Case Identification.....	23
Table 4.3: Use Case Description - Login (UC-001)	26
Table 4.4: Use Case Description - Submit Assignment (UC-010)	26
Table 4.5: Database Table.....	32

List of Tables

Table 1.1: Software Tools.....	7
2Table 1.2: Technology Stack	8
3Table 1.3: Hardware Tools	8
4Table 1.4: Project Schedule	12
5 Table 1.5: Project Cost Analysis.....	13
6 Table 1.6: Communication Plan	13
7 Table 1.7: Project Team.....	14
8Table 4.1: Actor Identification.....	23
9Table 4.2: Use Case Identification.....	23
10Table 4.3: Use Case Description - Login (UC-001).....	26
11Table 4.4: Use Case Description - Submit Assignment (UC-010).....	26
12Table 4.5: Database Table.....	32

List of Abbreviations and Acronyms

<i>Abbreviation</i>	<i>Full Form</i>
API	Application Programming Interface
ASP.NET	Active Server Pages .NET
BCrypt	Blowfish Cipher + Crypt hashing algorithm
CGPA	Cumulative Grade Point Average
CRUD	Create, Read, Update, Delete
CSS SCSS	Sassy Cascading Style Sheets
DTO	Data Transfer Object
ERD	Entity Relationship Diagram
ETB	Ethiopian Birr
HTML	HyperText Markup Language
IDE	Integrated Development Environment
JWT	JSON Web Token
MVC	Model-View-Controller
ORM	Object-Relational Mapping
PDF	Portable Document Format
REST	Representational State Transfer
RRT	Raras Technology
SQL	Structured Query Language
SSD	Solid State Drive
UML	Unified Modeling Language
USB	Universal Serial Bus
VM	Virtual Machine

Abstract

This documentation presents the design, development, and implementation of StudentPortal, a comprehensive web-based academic management system developed as part of the internship program at RRT Company. Higher education institutions in Ethiopia continue to rely on manual, paper-based systems for managing student records, grades, attendance, and fees, leading to significant challenges including data loss, processing delays, lack of transparency, and administrative inefficiency. The StudentPortal system addresses these challenges by providing a secure, efficient, and user-friendly platform that digitizes and streamlines key academic management processes. The system is built using ASP.NET Core 8 for the backend, Angular 21 for the frontend, and PostgreSQL 15 for the database, following object-oriented analysis and design principles. Key features include user authentication with role-based access control, student registration and approval workflows, department and course management, attendance tracking, assignment submission and grading, result entry and CGPA calculation, fee management in Ethiopian Birr, transcript generation, and notice management. The system supports three user roles: Admin, Faculty, and Student, each with tailored dashboards and functionality. The project followed a structured development methodology incorporating primary and secondary data collection, feasibility analysis, and iterative testing. The resulting system demonstrates significant improvements in operational efficiency, data accuracy, and transparency compared to manual processes, providing a scalable solution that aligns with Ethiopia's digital transformation goals in the education sector.

Chapter One: Introduction

1.1 Background of RRT Company

RRT Company is a forward-thinking technology solutions provider and software development firm based in Ethiopia. The company specializes in delivering innovative software solutions, IT consulting, and digital transformation services across multiple sectors including education, healthcare, finance, and government. RRT Company serves clients throughout Ethiopia and the broader East African region, with a strong commitment to addressing real-world challenges through cutting-edge technology.

Founded on the principles of innovation, quality, and client satisfaction, RRT Company has established itself as a reliable partner for organizations seeking to modernize their operations and embrace digital solutions. The company's core business areas include custom software development, enterprise application integration, cloud solutions, and IT infrastructure management.

RRT Company places a strong emphasis on talent development and knowledge transfer. The company's internship program provides students with hands-on experience in enterprise-level software development, exposure to industry best practices, and mentorship from experienced professionals. Through this program, RRT Company contributes to building the next generation of IT professionals in Ethiopia while delivering meaningful technology solutions to its clients.

The company's mission is to empower organizations through technology, and its vision is to become **a leading technology solutions provider in East Africa**. RRT Company's values include excellence, integrity, innovation, collaboration, and social responsibility.

1.2 Background of the Project

The StudentPortal project was initiated as part of the internship program at RRT Company to address the growing need for digital transformation in educational administration. Higher education institutions in Ethiopia continue to rely on manual, paper-based systems for managing

student records, grades, attendance, and fees. This approach has led to significant challenges including data loss, processing delays, lack of transparency, and administrative inefficiency.

Ethiopia's higher education sector has experienced rapid growth over the past decade, with increasing enrollment numbers putting additional strain on already overburdened administrative systems. Universities and colleges struggle to manage the volume of student data efficiently, leading to errors, delays, and frustration for both students and staff. The absence of a centralized digital system for academic management creates silos of information, makes cross-referencing difficult, and hampers decision-making processes.

The project aims to develop a comprehensive web-based student management system that automates key academic processes, provides real-time access to information, and improves overall operational efficiency. The StudentPortal system is designed to serve as a complete solution for academic management, covering everything from student registration to transcript generation.

This project aligns with RRT Company's mission to deliver innovative technology solutions that solve real-world problems and contribute to the digital transformation of key sectors in Ethiopia. It also aligns with the Ethiopian Ministry of Education's strategic plan for integrating information and communication technology (ICT) in education.

1.3 Statement of the Problem

The current manual system used by higher education institutions in Ethiopia suffers from several critical problems that significantly impact the quality of education delivery and administrative efficiency:

First, student records are maintained in paper files, which are vulnerable to loss, damage, and unauthorized access. When records are lost or damaged, institutions must invest significant time and resources in reconstruction, and in many cases, the data is permanently lost. This creates serious legal and academic risks for both the institution and the students.

Second, grade processing is slow and error-prone, with students often waiting weeks to receive their results. The manual calculation of grades and GPA is time-consuming and subject to human error, which can lead to incorrect academic records and student disputes. Faculty members spend excessive time on administrative tasks rather than focusing on teaching and research.

Third, attendance tracking is done manually using paper-based sign-in sheets, making it difficult to monitor student participation and enforce academic policies. There is no real-time visibility into attendance patterns, and generating attendance reports is a labor-intensive process.

Fourth, fee management lacks proper tracking and reporting mechanisms, leading to revenue loss and disputes. Institutions struggle to track outstanding balances, generate payment receipts, and reconcile accounts. The absence of a systematic fee tracking system creates financial management challenges and reduces institutional revenue.

Fifth, students cannot easily access their academic records or generate official transcripts, causing delays and frustrations. Requesting and processing transcripts through manual systems can take weeks, impacting students' ability to apply for jobs, scholarships, or further studies.

Sixth, administrators lack real-time visibility into key metrics such as student performance, attendance rates, and fee collection status. This lack of data-driven insight hampers decision-making and prevents institutions from identifying and addressing problems early.

Seventh, there is no centralized platform for communication between students, faculty, and administration. Important notices and announcements are communicated through physical notice boards or word of mouth, leading to missed information and communication gaps.

These challenges create significant inefficiencies, reduce transparency, and negatively impact the quality of education delivery. There is an urgent need for a comprehensive digital solution that addresses these problems and provides a modern, efficient, and transparent academic management system.

1.4 Objectives of the Project

1.4.1 General Objective

The general objective of this project is to design and develop a comprehensive web-based StudentPortal system that digitizes and streamlines academic management processes for Ethiopian higher education institutions, providing a secure, efficient, and user-friendly platform for students, faculty, and administrators.

1.4.2 Specific Objectives

The specific objectives of this project are:

- Objective 1: To develop a secure user authentication and role-based access control system using JWT tokens and BCrypt password hashing, ensuring data security and proper access management for all user types.
- Objective 2: To implement a student registration and approval workflow that includes pending approvals, email notifications, and role-based access, ensuring only authorized users can access the system.
- Objective 3: To create a department and course management module with auto-generation of academic year/semester records based on department duration, reducing manual setup and configuration effort.
- Objective 4: To develop attendance tracking and reporting functionality with percentage calculation and gatekeeper enforcement for assignments, ensuring academic policy compliance.
- Objective 5: To implement an assignment submission and grading system with PDF upload, file validation, and faculty grading capabilities, streamlining the assignment lifecycle.
- Objective 6: To create a result entry and CGPA calculation system that auto-calculates grades based on assignment, midterm, and final scores, ensuring accurate and consistent grade computation.

- Objective 7: To develop a fee management module in Ethiopian Birr (ETB) with payment tracking, transaction IDs, and status management, improving financial oversight and accountability.
- Objective 8: To implement transcript generation with professional PDF download and printing capabilities, enabling students to obtain their transcripts quickly and easily.
- Objective 9: To provide real-time dashboards for students, faculty, and administrators with key performance metrics, enabling data-driven decision-making.
- Objective 10: To implement a notice/announcement system with targeted messaging capabilities, ensuring effective communication across the academic community.

1.5 Methodology

1.5.1 Data Sources

The development of the StudentPortal system required comprehensive data collection to understand the requirements, challenges, and expectations of all stakeholders. Both primary and secondary data sources were utilized to ensure a thorough understanding of the problem domain.

1.5.1.1 Primary Data Source

Primary data was collected through direct interviews with university administrators, faculty members, and students. Structured questionnaires were distributed to gather specific requirements and feedback. On-site observations were conducted at selected higher education institutions to understand current workflows, identify bottlenecks, and document existing processes. Focus group discussions were held with key stakeholders to validate requirements and prioritize features. The primary data collection provided invaluable insights into the real-world challenges faced by users and informed the design and development of the StudentPortal system.

1.5.1.2 Secondary Data Source

Secondary data was collected from academic journals, research papers, technical documentation, and industry reports related to educational management systems. Relevant guidelines and policies from the Ethiopian Ministry of Education were reviewed to ensure compliance with national standards. Existing educational management systems, both local and international, were

analyzed to identify best practices and learn from previous implementations. Online resources, documentation, and tutorials for the technologies used (ASP.NET Core, Angular, PostgreSQL) were also reviewed to support the technical implementation.

1.5.2 Data Gathering Techniques

The following techniques were employed for data collection:

- Interviews: Semi-structured interviews were conducted with 15 stakeholders including 5 administrators, 5 faculty members, and 5 students. Each interview lasted 30-45 minutes and covered topics including current system workflows, pain points, desired features, and system expectations.
- Questionnaires: 50 questionnaires were distributed to students and faculty to gather quantitative data on system requirements and user expectations. The questionnaires used a Likert scale format to measure user preferences and priorities.
- Document Analysis: Existing documents including student registration forms, grade sheets, attendance records, and fee ledgers were analyzed to understand the current system and identify opportunities for automation.
- Observation: Direct observation of the current manual processes was conducted over a 2-week period to identify inefficiencies and bottlenecks. This included observing registration processes, grade processing, attendance taking, and fee collection.
- Focus Group Discussions: Two focus group sessions were conducted, one with administrators and one with faculty, to gather collective feedback and validate requirements. These sessions helped identify common themes and priorities across user groups.

1.5.3 Systems Analysis and Design

The system was analyzed and designed using the Object-Oriented Analysis and Design (OOAD) methodology. Unified Modeling Language (UML) was used to create diagrams representing the system's structure and behavior. The Waterfall software development lifecycle with Agile iterations was adopted, allowing for continuous stakeholder feedback. The development followed a phased approach: requirements gathering and analysis, system design, implementation, testing,

deployment, and maintenance. Key deliverables include use case diagrams, class diagrams, sequence diagrams, activity diagrams, and the database design.

1.5.4 Overview of the Project

The StudentPortal project involves the development of a web-based academic management system with three main user roles: Admin, Faculty, and Student. The system includes modules for user authentication and approval, department and course management, attendance tracking, assignment management, result entry and approval, fee management, and transcript generation. The project is developed using ASP.NET Core 8 for the backend, Angular 21 for the frontend, and PostgreSQL 15 for the database. The system is designed to be scalable, secure, and user-friendly, with a responsive interface that works on all devices. The project aims to provide a complete solution for academic management that can be deployed in Ethiopian higher education institutions.

1.5.5 Development Tools and Technology

The following tools and technologies were used in the development of the StudentPortal system:

Table 1.1: Software Tools

Tool	Purpose	Version
Microsoft Visual Studio 2022	IDE for backend development	17.0+
Visual Studio Code	Code editor for frontend	1.80+
PostgreSQL	Relational Database Management System	15.0+
pgAdmin	Database Administration Tool	7.0+
Git	Version Control System	2.40+
GitHub	Remote Repository Hosting	N/A
Swagger/OpenAPI	API Documentation and Testing	6.5+
Postman	API Testing and Development	10.0+
Node.js	JavaScript Runtime Environment	18.0+
npm	Package Manager for JavaScript	9.0+

Table 1.2: Technology Stack

Layer	Technology	Version	Purpose
Backend Framework	ASP.NET Core	8.0	Web API development
Frontend Framework	Angular (TypeScript)	21.0	Single Page Application
ORM	Entity Framework Core	8.0	Database access
Database	PostgreSQL	15.0	Data storage
Authentication	JWT + BCrypt	N/A	Security
PDF Generation	jsPDF + jspdf-autotable	N/A	Transcript generation
Styling	Angular Material + TailwindCSS/ SCSS	N/A	User Interface
Charts	ng2-charts (Chart.js)	N/A	Data visualization
Logging	Serilog	N/A	Application logging
API Testing	Swagger + Postman	N/A	Testing
Version Control	Git + GitHub	N/A	Source code management

Table 1.3: Hardware Tools

Tool	Specification
Development Machine	Intel Core i7 processor, 16GB RAM, 512GB SSD
Server	Cloud-based VM (4 vCPUs, 8GB RAM)
Network	High-speed internet connection

1.6 Scope of the Project

The StudentPortal system covers the following academic management functionalities: user registration and approval, department and course management, attendance tracking, assignment

submission and grading, result entry and approval, fee management in Ethiopian Birr (ETB), transcript generation, and notice management. The system supports three user roles: Admin, Faculty, and Student.

The system does not cover library management, hostel management, human resources management, or institutional financial accounting beyond student fee tracking. The system is designed for higher education institutions in Ethiopia and complies with the Ethiopian Ministry of Education guidelines.

In scope: User authentication and role-based access control; Student registration and approval workflow; Department and course management with auto-generation of academic periods; Attendance tracking with real-time percentage calculation; Assignment creation, submission, and grading; Result entry with auto CGPA calculation; Fee management and payment tracking in ETB; Transcript generation and PDF download; Notice and announcement system; Real-time dashboards for all roles.

Out of scope: Library management system; Hostel/accommodation management; Human resources management; Institutional financial accounting; Mobile applications (iOS/Android); SMS/email notification integration; Legacy system integration; Advanced analytics and machine learning.

1.7 Feasibility Study

1.7.1 Operational Feasibility

The StudentPortal system is operationally feasible as it is designed to be user-friendly and intuitive. The system uses web-based interfaces with familiar patterns that reduce the learning curve. Comprehensive user training will be provided to administrators, faculty, and students. The system's benefits, including real-time access to academic information, automated processes, and improved transparency, will encourage user adoption. The system does not disrupt existing workflows but enhances them, making it easier for users to accept the change.

1.7.2 Technical Feasibility

The StudentPortal system is technically feasible as all technologies used are mature, stable, and well-documented. The development team possesses the required skills in ASP.NET Core, Angular, PostgreSQL, and related technologies. The system architecture is designed to be scalable and maintainable. The required infrastructure is within reasonable cost and can be easily provisioned. PostgreSQL is a reliable and widely-used database that can handle the expected data volume. The web-based architecture allows for easy deployment and updates.

1.7.3 Behavioral/Political Feasibility

The StudentPortal system is behaviorally and politically feasible. The Ethiopian Ministry of Education is actively promoting digital transformation in education. University administrators, faculty, and students are increasingly receptive to technology solutions that improve efficiency and transparency. The project has strong support from key stakeholders including university leadership and the Ministry of Education. The system aligns with national education policies and priorities. Potential resistance to change can be addressed through training, communication, and phased implementation.

1.7.4 Economic Feasibility

The StudentPortal system is economically feasible. The development costs are reasonable and justified by the significant benefits. The system will reduce administrative overhead, save staff time, reduce paper and printing costs, improve fee collection, and enhance decision-making. The return on investment (ROI) is projected to be positive within the first year of operation. The total development cost is ETB 1,000,000 (approximately \$18,000 USD), which is a small investment compared to the expected annual savings and benefits.

1.8 Limitations of the Project

The StudentPortal system has several limitations that should be noted:

- The system requires reliable internet connectivity to function effectively, which may be a challenge in some rural areas of Ethiopia where internet infrastructure is still developing.

- The system currently only supports English language, although the Ministry of Education may require Amharic language support in the future.
- The system does not include native mobile applications (iOS/Android), although the web interface is fully responsive and works on mobile browsers.
- The system does not integrate with existing legacy systems such as HR and finance systems used by institutions.
- The system does not include advanced analytics or machine learning capabilities for predictive analysis.
- The system is designed for higher education institutions and may not be directly applicable to primary or secondary schools without significant modifications.
- The system does not include SMS or email notification capabilities for automatic alerts, although these can be added in future iterations.
- The system's performance may degrade with very large datasets (millions of records) without proper database optimization and scaling strategies.

1.9 Significance of the Project

The StudentPortal system provides significant benefits to all stakeholders. Students gain immediate access to their academic records, grades, assignments, attendance, and fee information, empowering them to take ownership of their education. Faculty can efficiently manage attendance, assignments, and results, reducing administrative burden and allowing more time for teaching and research. Administrators have full visibility and control over the academic process, enabling data-driven decision-making.

The institution benefits from streamlined operations, reduced administrative overhead, improved data accuracy, enhanced transparency, and better regulatory compliance. The Ministry of Education gains a model system that can be replicated across other institutions in Ethiopia, contributing to national digital transformation goals. The project also contributes to the body of knowledge on educational technology and provides valuable insights for future similar projects.

1.10 Task and Schedule

The project was executed according to the following schedule:

Table 1.4: Project Schedule

Task	Duration	Start Week	End Week	Milestone
Requirements Gathering	2 weeks	Week 1	Week 2	Requirements Document
System Analysis	2 weeks	Week 3	Week 4	Analysis Document
UI/UX Design	2 weeks	Week 5	Week 6	Design Prototype
Database Design	1 week	Week 7	Week 7	Database Schema
Backend Development	4 weeks	Week 8	Week 11	API Completion
Frontend Development	4 weeks	Week 8	Week 11	UI Completion
API Integration	1 week	Week 12	Week 12	Integrated System
Unit Testing	1 week	Week 13	Week 13	Test Results
Integration Testing	1 week	Week 14	Week 14	Test Report
User Testing	1 week	Week 15	Week 15	User Feedback
Deployment	1 week	Week 16	Week 16	Live System
Documentation	2 weeks	Week 15	Week 16	Final Report

1.11 Cost Analysis of the Project

Table 1.5: Project Cost Analysis

Item	Cost (ETB)	Cost (USD)
Development Hardware	150,000	2,700
Software Licenses	50,000	900
Server Hosting (Annual)	100,000	1,800
Database Hosting (Annual)	80,000	1,440
Developer Salaries	500,000	9,000
Training and Support	100,000	1,800
Miscellaneous	20,000	360
Total	1,000,000	18,000

1.12 Communication Plan

Table 1.6: Communication Plan

Stakeholder	Frequency	Format	Responsibility
Project Sponsor (RRT Company)	Monthly	Progress Report	Project Manager
Steering Committee	Quarterly	Presentation	Project Manager
Development Team	Daily	Daily Stand-up	Team Lead
Users (Students/Faculty)	Bi-weekly	Meeting	Business Analyst
Ministry of Education	Quarterly	Presentation	Project Manager
RRT Company Management	Monthly	Report	Project Manager
Internship Supervisor	Weekly	Meeting	Project Manager

1.13 Project Team Composition

Table 1.7: Project Team

Role	Name	Responsibility
Project Manager	[Name]Mr. Getenet A	Overall project coordination, stakeholder management
Lead Developer	Mut Gatkek	Backend architecture, API development
Frontend Developer	[Name]Mut Gatkek	UI/UX implementation, Angular development
Database Administrator	[Name]Mut Gatkek	Database design, optimization, maintenance
Quality Assurance	[Name]Mut Gatkek	Testing, quality control, bug tracking
Business Analyst	[Name]Mut Gatkek	Requirements gathering, documentation
UI/UX Designer	[Name]Mut Gatkek	User interface design, prototyping

Chapter Two: Existing System

2.1 Description of the Existing System

The existing system for academic management at higher education institutions in Ethiopia is predominantly manual and paper-based. Student records are maintained in physical files and register books. Grade processing is done using paper-based grade sheets and manual calculations. Attendance is tracked using sign-in sheets or paper registers. Fee management relies on manual receipts and ledger books. Communication is handled through physical notice boards and word-of-mouth.

When a student enrolls at an institution, their personal and academic information is recorded in a physical registration form and entered into a paper-based register. Throughout their academic journey, their grades are recorded on paper grade sheets by faculty members at the end of each semester. Attendance is taken at the beginning of each class session using paper sign-in sheets. Fee payments are recorded manually in fee ledgers, and receipts are issued by hand.

Students must physically visit administrative offices to request transcripts, check their grades, or inquire about their fee status. Faculty members must submit their grade sheets to the registrar's office, where staff manually enter the data and calculate GPAs. Administrators compile reports by manually aggregating data from various paper-based sources.

2.2 Problems of the Existing System

2.2.1 Performance Issues

The manual system suffers from significant performance issues. Grade processing takes weeks to complete due to the manual calculation and data entry involved. Students often wait extended periods to receive their results, which impacts their ability to plan for the next semester. Administrative tasks such as generating reports and transcripts are time-consuming and require multiple manual steps. The system cannot handle peak loads efficiently, leading to bottlenecks during registration and grade processing periods.

2.2.2 Security and Control Issues

Paper-based records are highly vulnerable to security threats. Files can be stolen, damaged by water or fire, or accessed by unauthorized individuals. There is no audit trail to track who accessed or modified records. Grade manipulation is difficult to detect in a paper-based system. Confidential student information is at risk of exposure due to improper storage and handling of physical documents.

2.2.3 Economic Issues

The manual system incurs significant ongoing costs including paper, printing, storage space, and additional administrative staff. The cost of physical storage for paper records is substantial and increases over time. Errors in manual processing lead to rework and additional costs. The inefficiency of the system reduces staff productivity, effectively increasing the cost per student processed.

2.2.4 Data Storage Issues

Physical data storage is limited by available space and is susceptible to deterioration over time. Searching for specific records is slow and labor-intensive. There is no backup mechanism, meaning data loss is permanent. Redundant copies of records are rarely maintained, increasing the risk of data loss. Data consistency is difficult to maintain when the same information is recorded in multiple physical locations.

2.2.5 Efficiency Issues

The manual system is inherently inefficient. Faculty members spend excessive time on administrative tasks such as recording attendance and computing grades, time that could be better spent on teaching and research. Students must physically visit offices for routine inquiries, wasting time and creating long queues. Information sharing between departments is slow and often requires physical document transfer. Real-time decision-making is impossible with a paper-based system.

2.3 Actors in the Existing System

The following actors are involved in the existing system:

- Students: Enroll in courses, attend classes, submit assignments, take exams, check results, request transcripts, and pay fees.
- Faculty Members: Teach courses, record attendance, create assignments, grade student work, and submit results to the registrar.
- Administrators/Registrars: Process student registrations, manage academic records, process transcripts, and compile institutional reports.
- Accounting Staff: Record fee payments, issue receipts, manage fee ledgers, and generate financial reports.
- Management: Oversee operations, review reports, and make institutional decisions.

2.4 Business Rules of the System

The existing system operates under the following business rules:

- A student must complete the registration process and pay all required fees before being admitted to classes.
- Attendance below 75% results in the student being barred from sitting for examinations.
- Grades are calculated based on assignment scores (20%), midterm exam (30%), and final exam (50%).
- A student must pass all required courses in a semester to advance to the next year.
- Transcripts can only be issued after the student's final results have been approved by the academic committee.
- Fee payments must be completed before results or transcripts are released.
- Only authorized administrative staff can modify student records.
- Faculty members can only record grades for courses they are assigned to teach.

2.5 Reports Generated in the Existing System

The existing system generates the following reports manually:

- Student Registration Report: Lists all registered students for a given semester.

- Grade Report: Contains student grades for all courses in a semester.
- Attendance Report: Shows student attendance records for each course.
- Fee Collection Report: Summarizes fee payments received.
- Transcript: Official record of a student's academic history.
- Enrollment Statistics: Number of students enrolled by department and year.

2.6 Alternative Options to Address Problems

Several alternative options were considered to address the problems identified in the existing system:

Option 1: Continue with the manual system but improve paper-based processes. This option was rejected because it does not address the fundamental issues of data loss, inefficiency, and lack of real-time access.

Option 2: Purchase an off-the-shelf commercial student management system. This option was considered but found to be too expensive for most Ethiopian institutions and may not align with local requirements and regulations.

Option 3: Develop a custom web-based student management system. This option was selected as the most viable solution because it allows for customization to meet specific institutional requirements, is cost-effective, and provides the flexibility to add features as needed. This is the approach adopted in the StudentPortal project.

Option 4: Outsource the development to an external software company. While feasible, this option was deemed less desirable due to higher costs and reduced control over the development process. The internship-based development approach provides a cost-effective solution while building local capacity.

Chapter Three: Proposed System

3.1 Overview of the Proposed System

The proposed StudentPortal system is a comprehensive web-based academic management platform designed to address all the problems identified in the existing system. The system provides a modern, secure, and efficient solution for managing student records, attendance, assignments, grades, fees, and communication.

The system is built on a three-tier architecture consisting of a presentation layer (Angular frontend), a business logic layer (ASP.NET Core backend), and a data layer (PostgreSQL database). This architecture ensures separation of concerns, maintainability, and scalability.

The system supports three user roles with distinct capabilities: Administrators who manage the entire system including user approval, departments, courses, and result approval; Faculty members who manage attendance, create and grade assignments, and enter results; and Students who register, submit assignments, view results, pay fees, and generate transcripts.

3.2 Functional Requirements

The following functional requirements are implemented in the proposed system:

- FR-001: The system shall allow users to register with email, full name, and password.
- FR-002: The system shall require administrator approval before a new user can access the system.
- FR-003: The system shall authenticate users using JWT tokens with BCrypt password hashing.
- FR-004: The system shall support three user roles: Admin, Faculty, and Student.
- FR-005: The system shall allow administrators to create and manage academic departments.
- FR-006: The system shall allow administrators to create and manage courses with auto-generated academic periods.

- FR-007: The system shall allow administrators to assign faculty to departments and courses.
- FR-008: The system shall allow faculty to mark attendance for their assigned courses.
- FR-009: The system shall calculate attendance percentages in real-time.
- FR-010: The system shall allow faculty to create assignments with due dates and maximum scores.
- FR-011: The system shall allow students to submit assignments as PDF files.
- FR-012: The system shall enforce a 75% attendance gatekeeper for assignment submissions.
- FR-013: The system shall allow faculty to grade assignments with scores.
- FR-014: The system shall allow faculty to enter midterm and final exam scores.
- FR-015: The system shall auto-calculate total scores and grades based on predefined weights.
- FR-016: The system shall require administrator approval before results are visible to students dashboard/profile.
- FR-017: The system shall calculate CGPA automatically based on approved results.
- FR-018: The system shall manage student fees in Ethiopian Birr (ETB) with payment tracking.
- FR-019: The system shall generate professional transcripts in PDF format.
- FR-020: The system shall provide role-based dashboards with key performance metrics.
- FR-021: The system shall support notice/announcement creation with targeted messaging.
- FR-022: The system shall allow users to update their profiles and change passwords.

3.3 Non-Functional Requirements

The following non-functional requirements apply to the proposed system:

- NFR-001: The system shall respond to user requests within 3 seconds under normal load.
- NFR-002: The system shall be available 99.5% of the time during operational hours.

- NFR-003: The system shall support at least 500 concurrent users without performance degradation.
- NFR-004: All data in transit shall be encrypted using TLS/SSL.
- NFR-005: All passwords shall be stored using BCrypt hashing with appropriate salt rounds.
- NFR-006: The system shall maintain audit logs for all critical operations.
- NFR-007: The system shall perform daily automated backups of the database.
- NFR-008: The system shall be responsive and work on desktop, tablet, and mobile browsers.
- NFR-009: The system shall follow WCAG 2.1 accessibility guidelines.
- NFR-010: The system shall be designed for horizontal scalability to support future growth.

Chapter Four: System Design

4.1 System Design Using Object Oriented Paradigm

The StudentPortal system is designed using the Object-Oriented paradigm, which models the system as a collection of interacting objects. Each object encapsulates data and behavior, promoting modularity, reusability, and maintainability. The design follows the principles of encapsulation, inheritance, and polymorphism to create a robust and flexible system architecture.

4.1.1 Design Goals

The key design goals for the StudentPortal system are:

- **Modularity:** The system is divided into independent modules (authentication, attendance, assignments, results, fees, transcripts) that can be developed, tested, and maintained separately.
- **Scalability:** The architecture supports horizontal scaling through load balancing and database replication.
- **Security:** The system implements industry-standard security practices including JWT authentication, BCrypt password hashing, role-based access control, and input validation.
- **Usability:** The user interface is designed to be intuitive, responsive, and accessible across all device types.
- **Maintainability:** The codebase follows clean architecture principles with clear separation of concerns, making it easy to modify and extend.
- **Performance:** The system is optimized for fast response times through efficient database queries, caching strategies, and lazy loading.

4.2 Use Case Diagram

4.2.1 Actor and Use Case Identification

The following actors and use cases are identified for the StudentPortal system:

Table 4.1: Actor Identification

Actor	Type	Description
Admin	Primary	Manages the entire system including users, departments, courses, and result approval
Faculty	Primary	Teaches courses, manages attendance, creates/grades assignments, and enters results
Student	Primary	Registers, attends classes, submits assignments, views results, pays fees

Table 4.2: Use Case Identification

Use Case ID	Use Case Name	Primary Actor
UC-001	Login	All Users
UC-002	Register Account	Student
UC-003	Approve User	Admin
UC-004	Manage Departments	Admin
UC-005	Manage Courses	Admin
UC-006	Assign Faculty to Course	Admin
UC-007	Enroll Student in Course	Admin
UC-008	Mark Attendance	Faculty

UC-009	Create Assignment	Faculty
UC-010	Submit Assignment	Student
UC-011	Grade Assignment	Faculty
UC-012	Enter Results	Faculty
UC-013	Approve Results	Admin
UC-014	Generate Transcript	Student
UC-015	Manage Fees	Admin
UC-016	Pay Fees	Student
UC-017	Send Notice	Admin/Faculty
UC-018	View Dashboard	All Users
UC-019	Update Profile	All Users
UC-020	Change Password	All Users
UC-021	View Results	Student
UC-022	View Attendance	Student
UC-023	Generate Report	Admin

4.3 Use Case Diagram

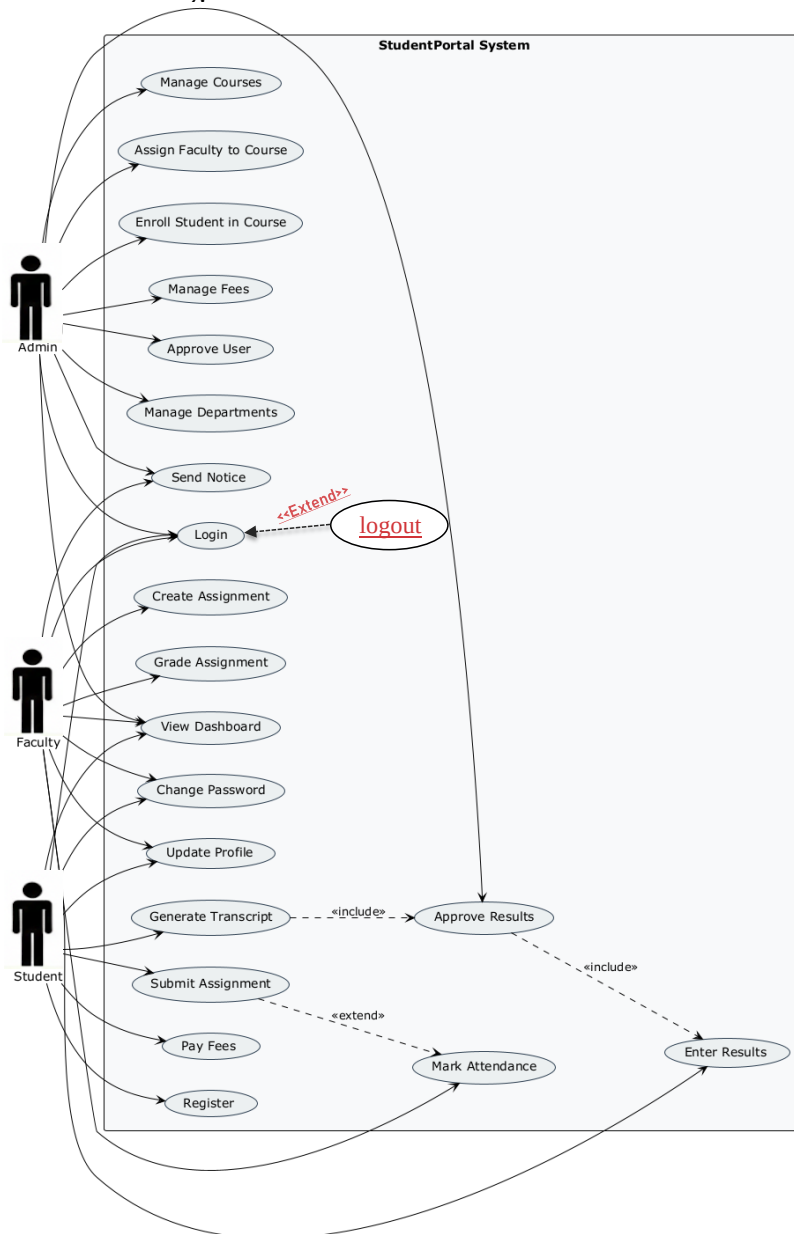


Figure 4.1: Use Case Diagram

Table 4.3: Use Case Description - Login (UC-001)

Field	Description
Use Case ID	UC-001
Use Case Name	Login
Actor	Admin, Faculty, Student
Precondition	User has a registered and approved account
Main Flow	1. User enters email and password 2. System validates credentials 3. System checks account approval status 4. System generates JWT token 5. User is redirected to role-specific dashboard
Alternative Flow	Invalid credentials: System shows error message Pending approval: System shows pending message
Postcondition	User is authenticated and redirected to dashboard

Table 4.4: Use Case Description - Submit Assignment (UC-010)

Field	Description
Use Case ID	UC-010
Use Case Name	Submit Assignment
Actor	Student
Precondition	Student is logged in, enrolled in course, has $\geq 75\%$ attendance
Main Flow	1. Student selects assignment 2. Student uploads PDF file (< 10MB) 3. System validates file type and attendance 4. System saves submission 5. Student receives confirmation
Alternative Flow	Attendance < 75%: Submission blocked Invalid file type: Error message File too large: Error message
Postcondition	Assignment is submitted and recorded in the system

4.4 Sequence Diagrams

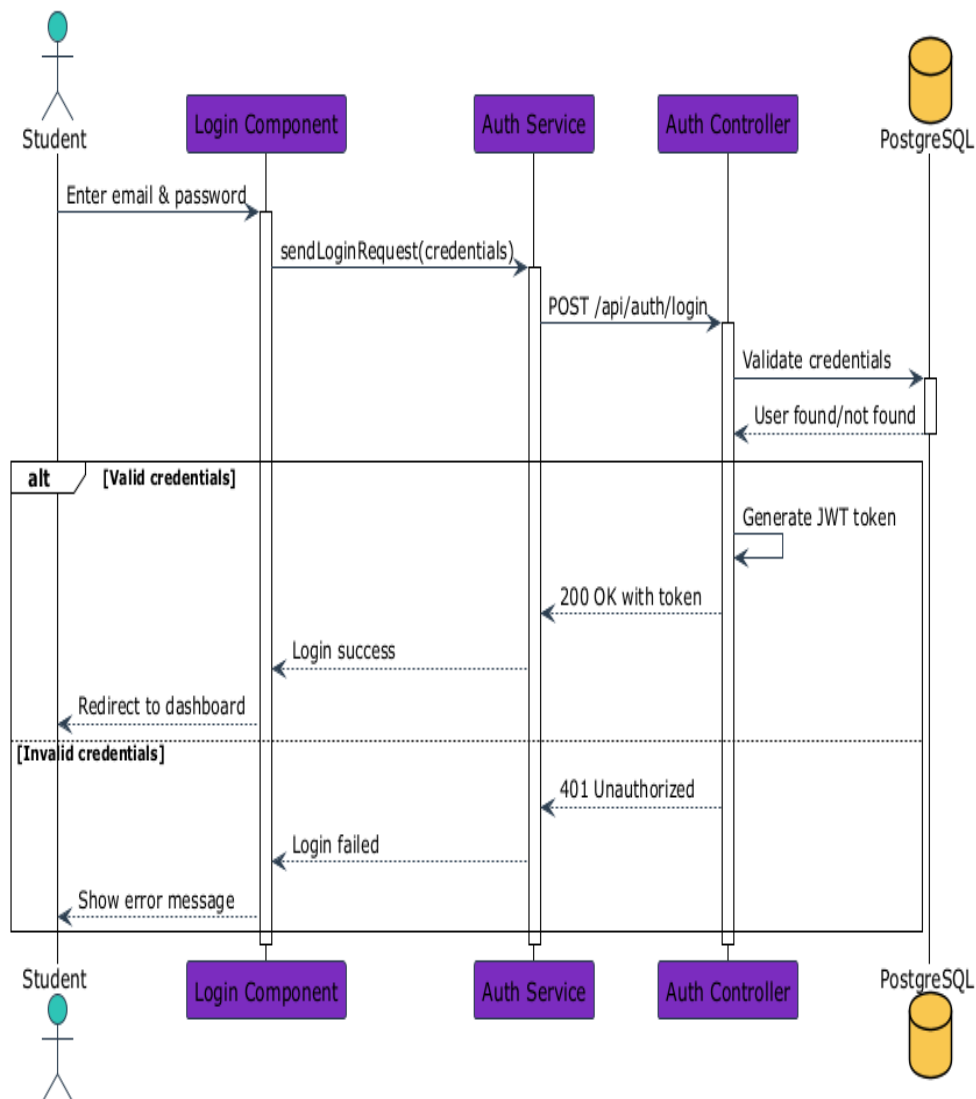


Figure 4.2: Login Sequence Diagram

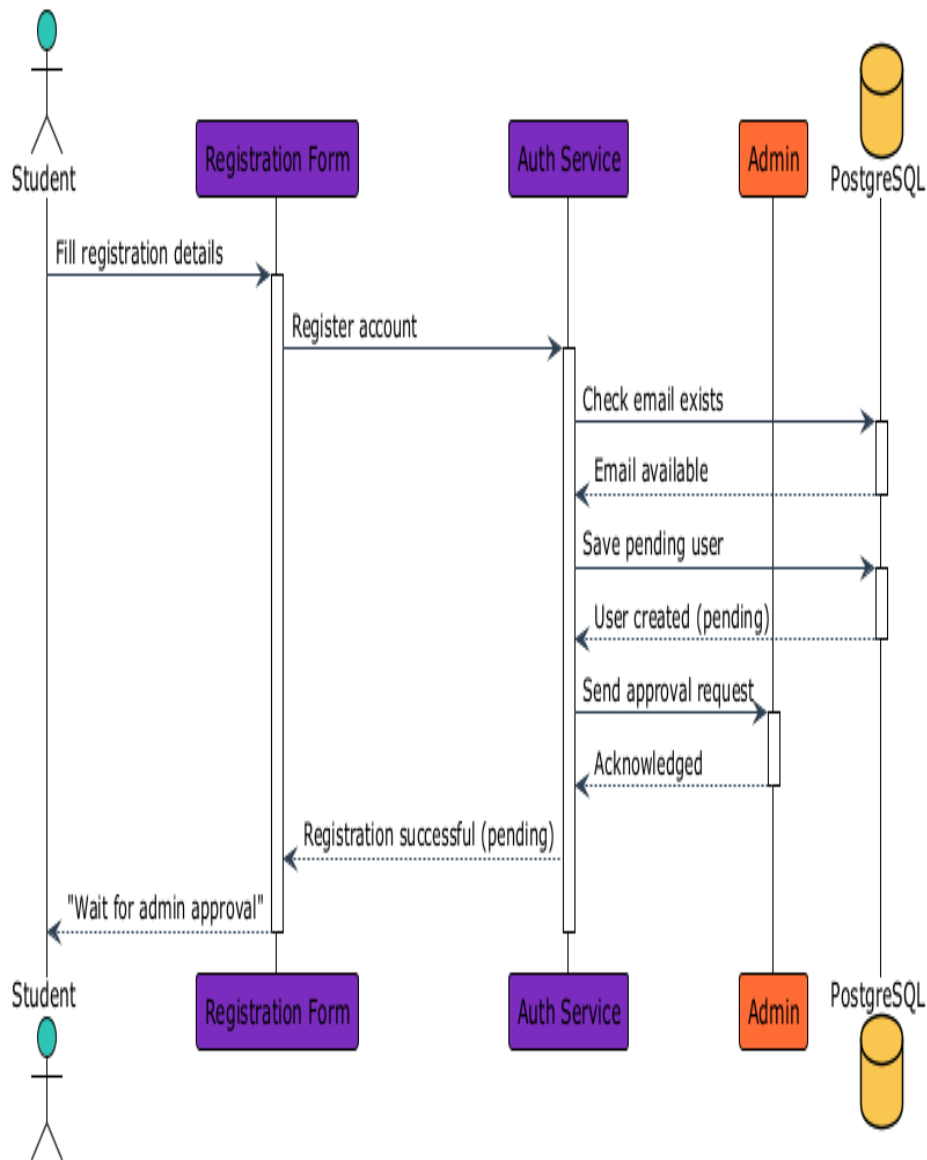


Figure 4.3: Account Registration Sequence Diagram

4.5 Activity Diagrams

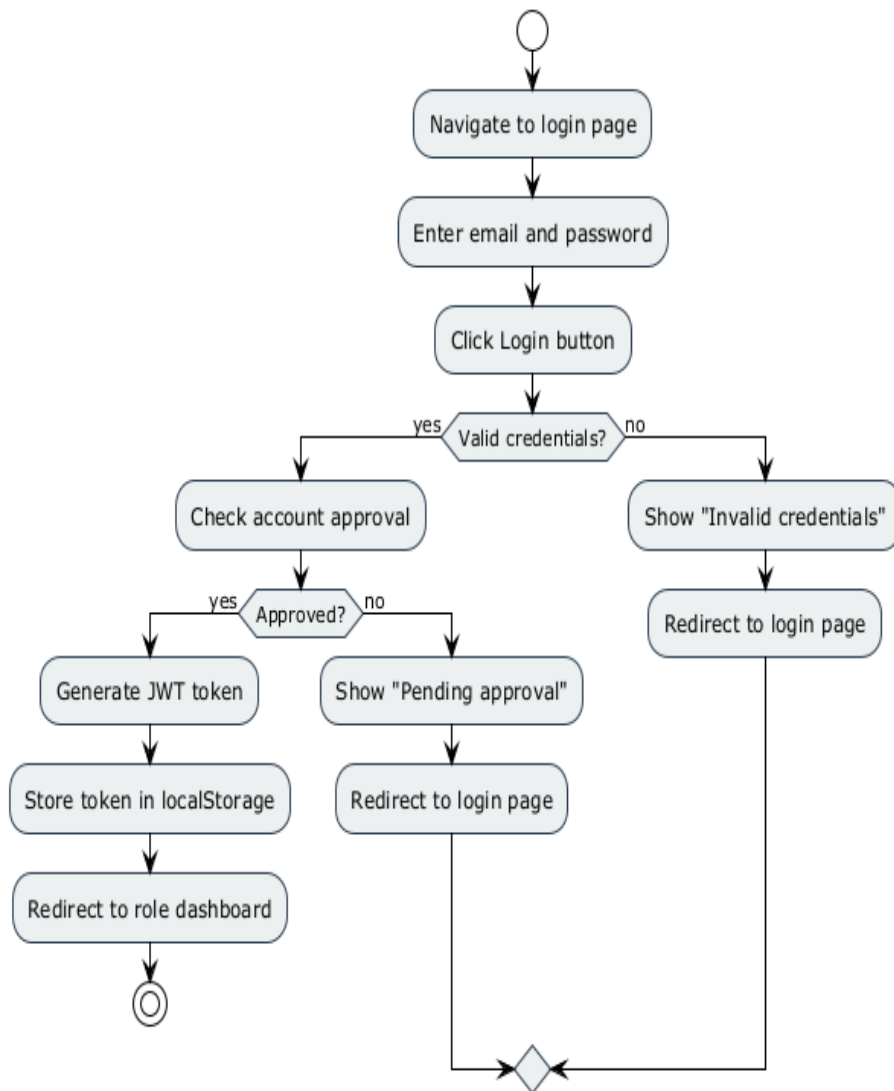


Figure 4.4: Login Activity Diagram

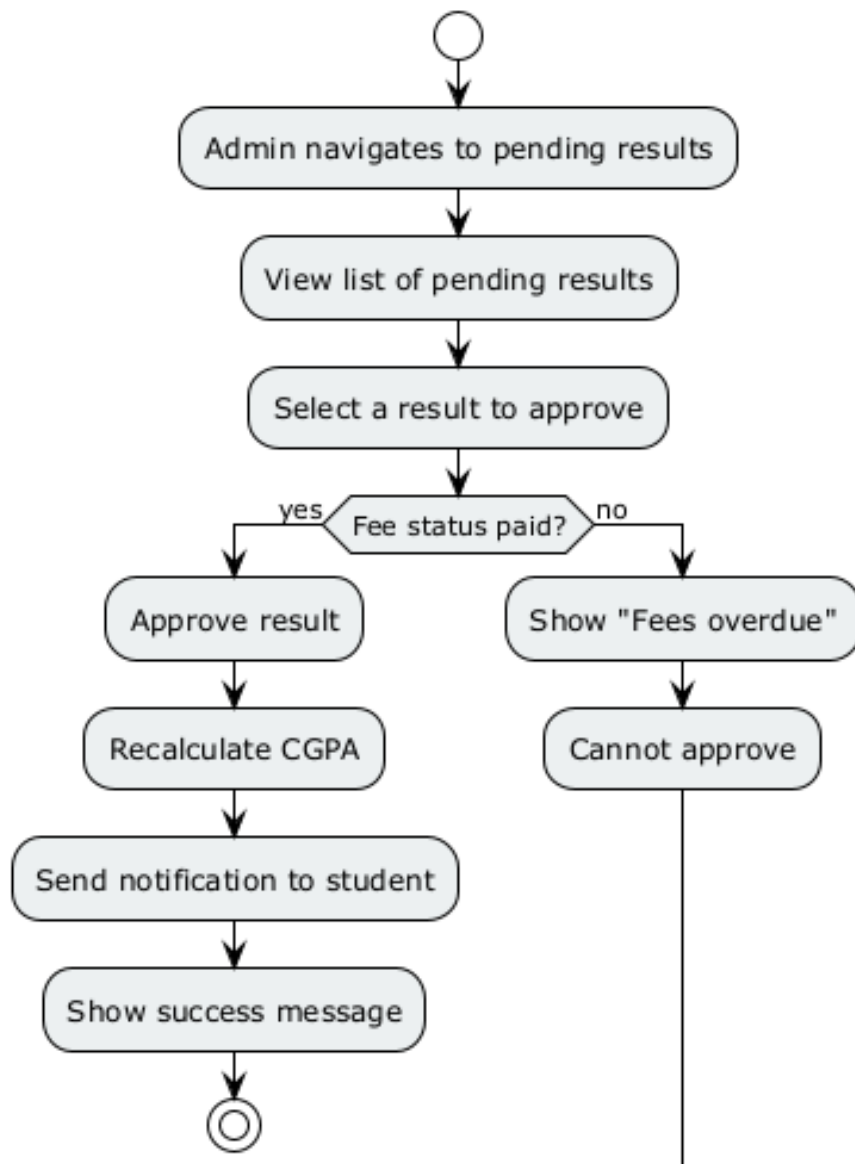


Figure 4.5: Result Approval Activity Diagram

4.6 Class Diagram

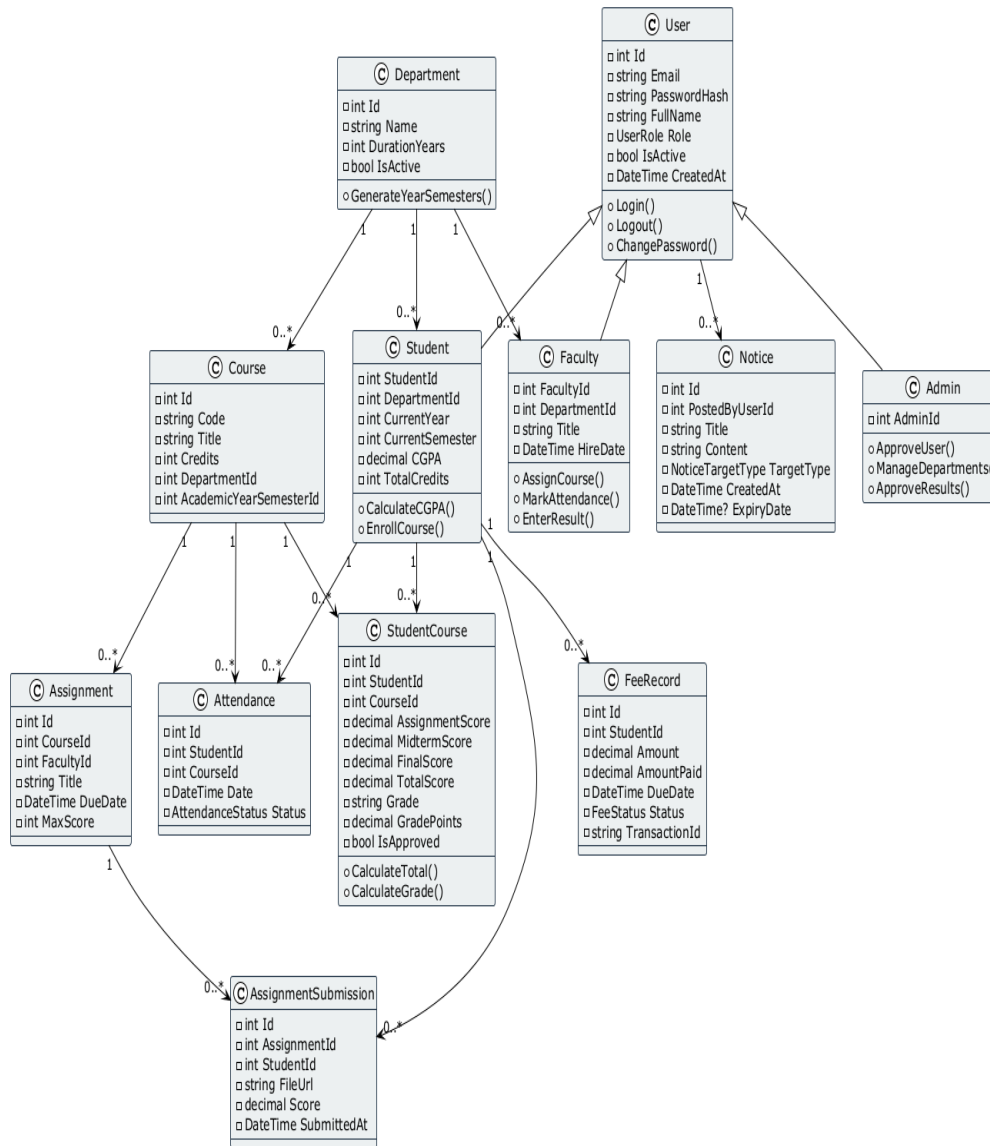


Figure 4.6: Class Diagram

4.7 Persistent Modeling / Database Design

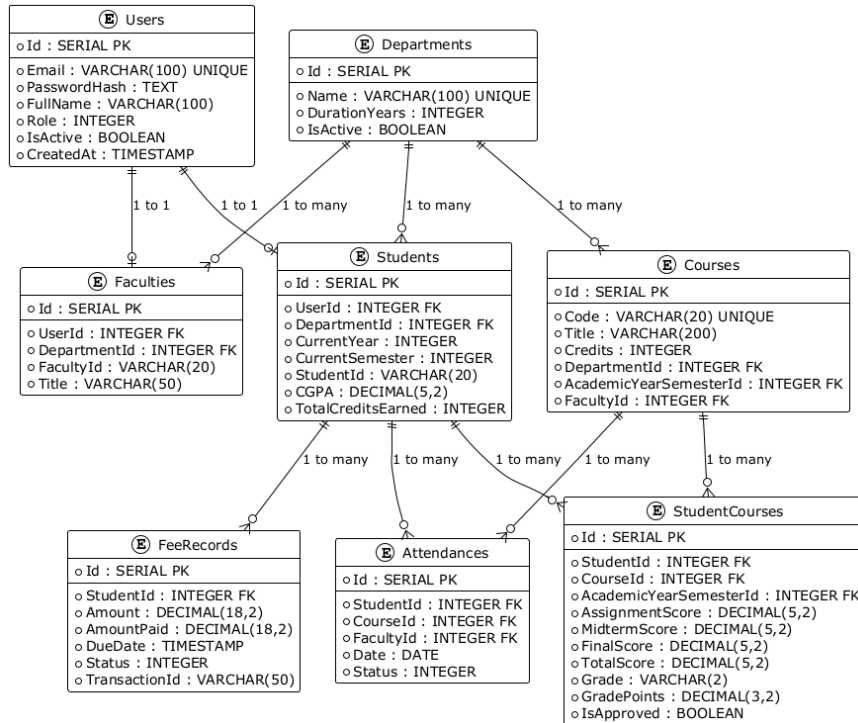


Figure 4.7: Entity Relationship Diagram (ERD)

Table 4.5: Database Table

Table Name	Description	Key Fields
Users	All user accounts	Id (PK), Email, PasswordHash, FullName, Role, IsActive
Students	Student-specific data	Id (PK), UserId (FK), DepartmentId (FK), CGPA, TotalCreditsEarned
Faculties	Faculty-specific data	Id (PK), UserId (FK), DepartmentId (FK), Title

Departments	Academic departments	Id (PK), Name, DurationYears, IsActive
Courses	Course catalog	Id (PK), Code, Title, Credits, DepartmentId (FK), FacultyId (FK)
StudentCourses	Enrollment & grades	Id (PK), StudentId (FK), CourseId (FK), Grade, IsApproved
Attendances	Attendance records	Id (PK), StudentId (FK), CourseId (FK), Date, Status
FeeRecords	Fee payments	Id (PK), StudentId (FK), Amount, AmountPaid, Status

4.8 Component Diagram

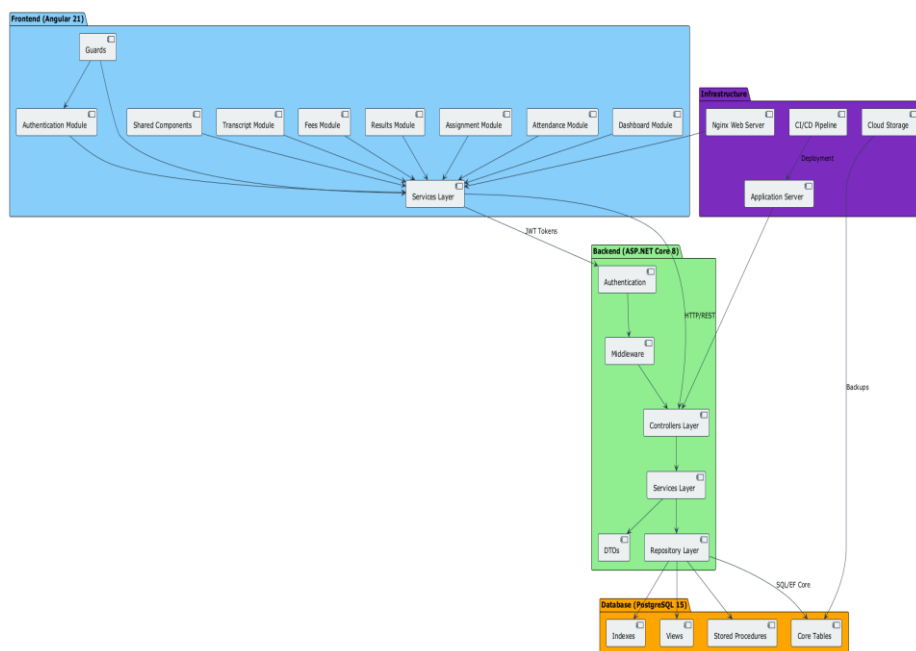


Figure 4.8: Component Diagram

4.9 Deployment Diagram

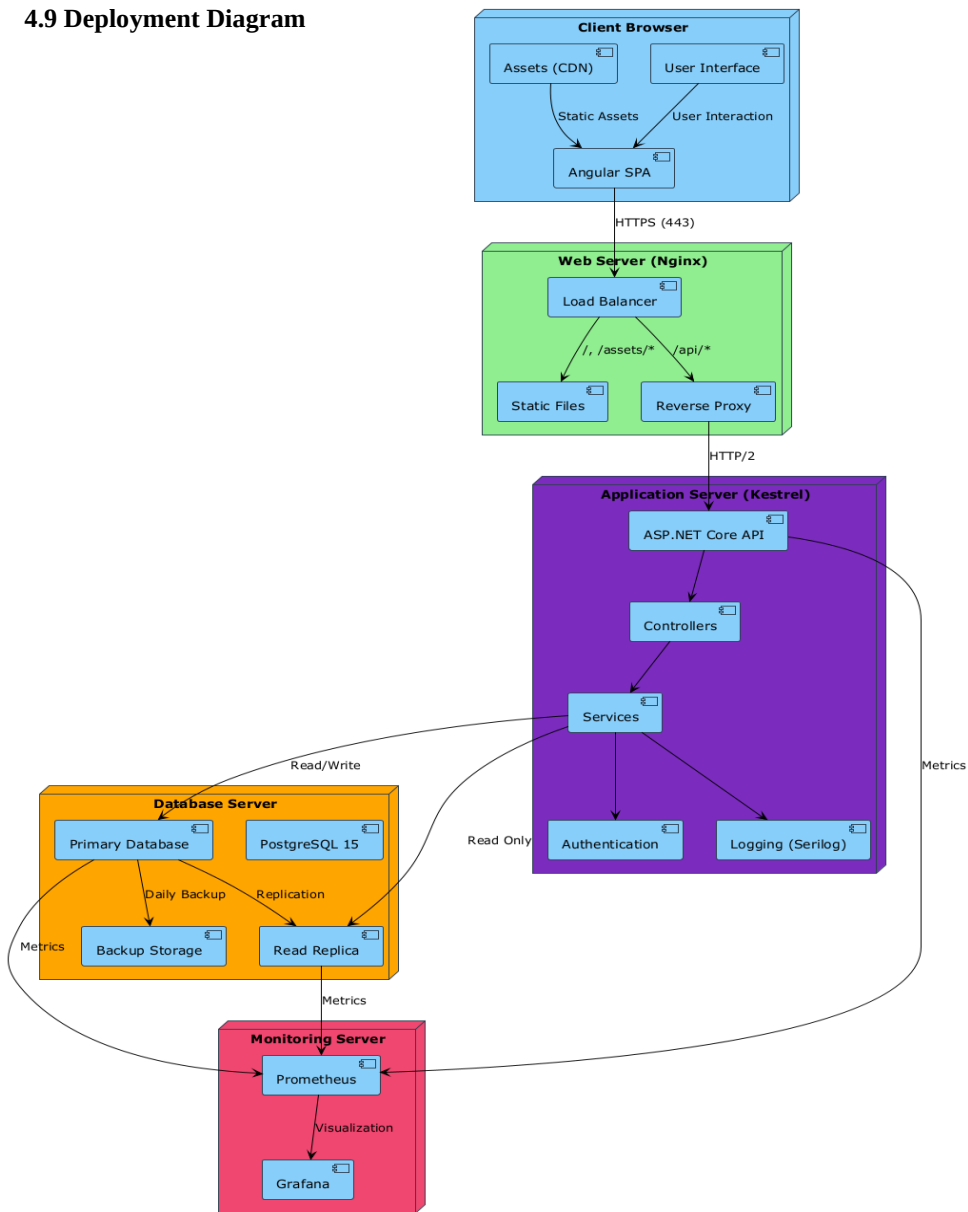
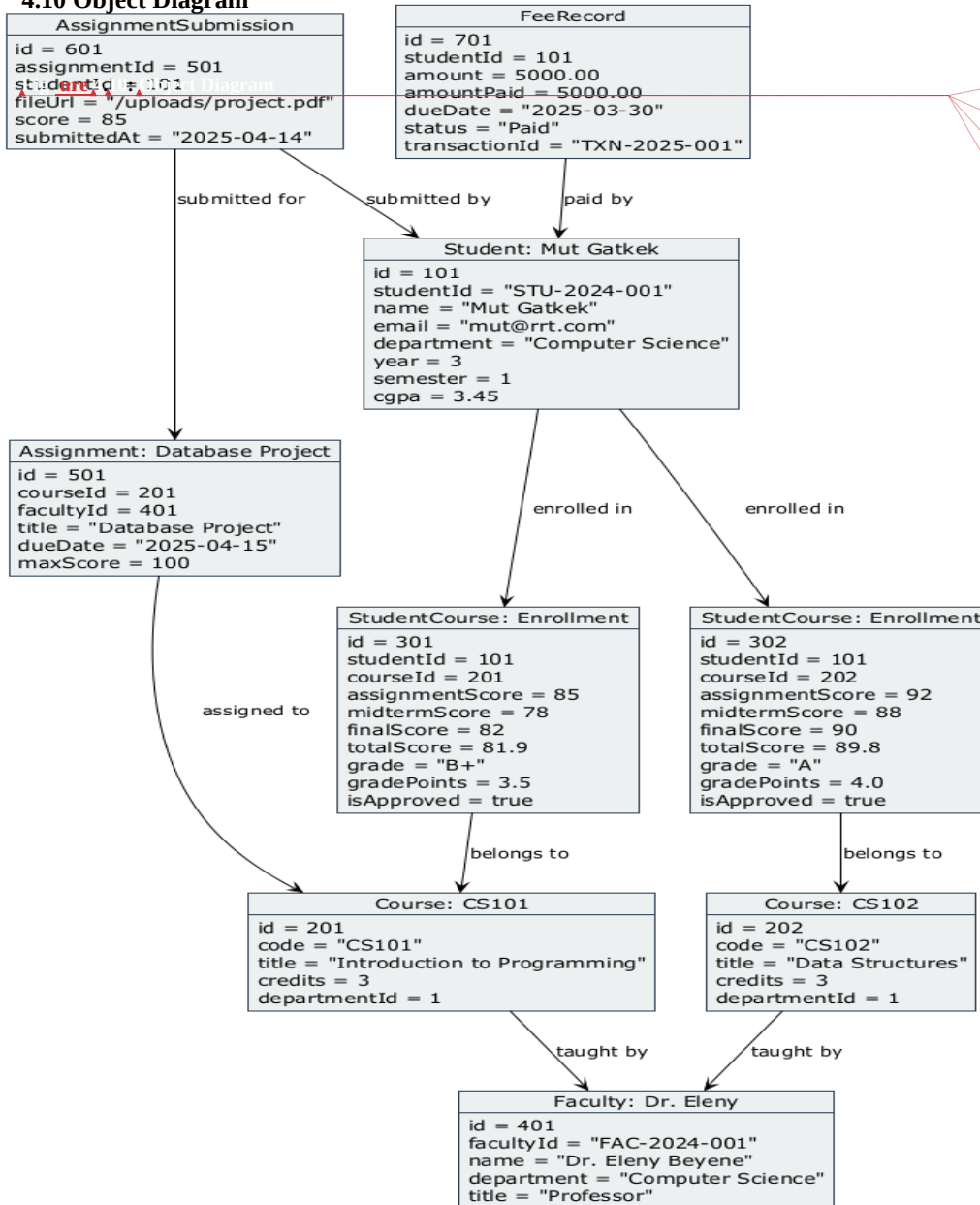


Figure 4.9: Deployment Diagram

4.10 Object Diagram



Formatted: Font color: Background 1

Formatted: Font color: Background 1

Formatted: Font color: Background 1

Formatted: Font color: Background 1

Figure 4.10: Object Diagram

RRT Company Internship Program 2026

Chapter Five: System Implementation

5.1 Introduction

Formatted: Font: Bold

The implementation phase is the most crucial phase in which it transforms the design and analysis of the system into a tangible system by writing the code to the system to be developed and make it operational and applicable by testing and debugging the functionalities that are done. This phase involves the construction of the actual project result. During this phase that the project becomes visible to outsiders, to whom it may appear that the project has just begun. This makes the implementation stage more essential step to develop the required system. So, it is the most vital and necessary stage in achieving a successful system and in giving the users confidence that the new system will work and be effective by testing the system that is already implemented. In this phase, the production system is installed, initial user training is completed, user documentation is delivered, and the post implementation review meeting is held. When this phase is completed, the application is in steady state production. Once the system is in steady-state production, it is reviewed to ensure that we met all of the goals in the project plan for a satisfactory result. The result of this phase consists of source code, together with documentation to make the code more readable. This is what we call software implementation. The purpose of these activities is to convert the final physical system specification into working model with reliable software and hardware, document the work that has been done, and provide help for current and future users and take care of the system.

5.2 Sample code

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { AuthService } from '../core/services/auth.service';
import { NotificationService } from '../core/services/notification.service';
@Component({
```

```

selector: 'app-login',
templateUrl: './login.html',
styleUrls: ['./login.scss'],
standalone: false
})

export class LoginComponent {

  loginForm: FormGroup;

  loading = false;

  hidePassword = true;

  constructor(

    private fb: FormBuilder,

    private auth: AuthService,

    private router: Router,

    private notification: NotificationService

  ) {

    this.loginForm = this.fb.group({

      email: ['', [Validators.required, Validators.email]],

      password: ['', [Validators.required, Validators.minLength(6)]]

    });

  }

  onSubmit(): void {

    if (this.loginForm.invalid) return;

    this.loading = true;

    this.auth.login(this.loginForm.value).subscribe({

```

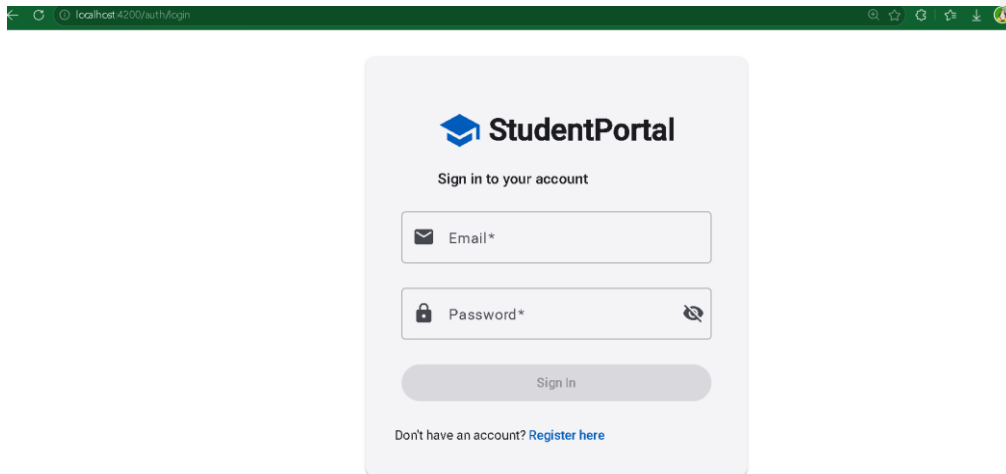
```

next: (res) => {
  this.loading = false;
  if (!res.isActive) {
    this.auth.logout();
    this.router.navigate(['/auth/pending-approval']);
    return;
  }
  this.notification.success(`Welcome back, ${res.fullName}!`);
  this.redirectByRole(res.role);
},
error: (err) => {
  this.loading = false;
  const message = err.error?.message || err.error?.error || 'Invalid credentials';
  this.notification.error(message);
}
});
}

private redirectByRole(role: string): void {
  switch (role) {
    case 'Admin': this.router.navigate(['/admin']); break;
    case 'Faculty': this.router.navigate(['/faculty']); break;
    case 'Student': this.router.navigate(['/student']); break;
    default: this.router.navigate(['/']);
  } } }

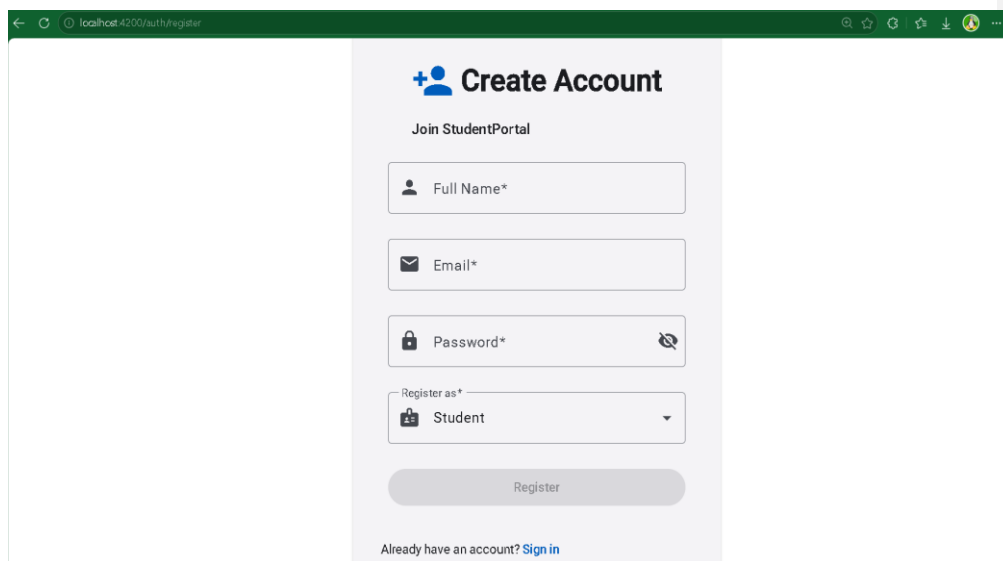
```

5.3 User interface



The screenshot shows a web browser window with the address bar displaying 'localhost:4200/auth/login'. The main content area features a light gray card with the 'StudentPortal' logo at the top. Below the logo, the text 'Sign in to your account' is centered. The card contains two input fields: 'Email*' with an envelope icon and 'Password*' with a lock icon and a toggle for visibility. A 'Sign In' button is positioned below these fields. At the bottom of the card, there is a link that says 'Don't have an account? Register here'.

Figure 5.1 User interface home/login page



The screenshot shows a web browser window with the address bar displaying 'localhost:4200/auth/register'. The main content area features a light gray card with a '+ Create Account' header. Below the header, the text 'Join StudentPortal' is centered. The card contains four input fields: 'Full Name*' with a person icon, 'Email*' with an envelope icon, 'Password*' with a lock icon and a toggle for visibility, and 'Register as*' with a dropdown menu currently showing 'Student'. A 'Register' button is positioned below these fields. At the bottom of the card, there is a link that says 'Already have an account? Sign in'.

Figure 5.2 User interface Create Account page

5.4 Testing

The test plan is designed to analyze the logic used in the implementation of the Student-Portal system. The tests will allow us to ensure correct program flow, and to determine the error-handling capability of the system. Once coding has begun, the testing process can be beginning and proceed in parallel. As each program module is produced, it can be tested individually, as a part of larger program, and then as a part of larger system.

Unit testing: Unit test is a way of testing each of the system functionality independently. Accordingly, the team has tested each one of the major activities and the rest accompanying activities independently using different user input, different login mechanisms and any technique of fault finding so that an incorrect functioning of the activities was corrected at the right time.

Integration Testing: Integration testing is testing in which a group of components are combined to produce output. Also, the interaction between software and hardware is tested in integration testing if software and hardware components have any relation.

System testing: It is the final step of testing. In this system tested the entire system as a whole with all forms, code, modules. In this we tested all the functionalities in the System. All errors in the forms, functions, modules have been tested. Finally, System testing ensures that the entire integrated software system meets the desired requirements. It tests a configuration to ensure known and predictable results. 5.5 User manual preparation User manual preparation since the system is web-based application everything important for the user will be explained and implemented while preparation of short training document when the system is deployed.

5.6 Training

Training manuals serve the important purpose of providing a consistent way to communicate instructions to employees about how to perform essential functions of their jobs by using the system. To be effective, training manuals is based on functionalities and instructional objectives. Information should be provided about the processes and procedures employees are required to follow as well as tasks that form the basis of the job, they are charged with performing.

5.7 Start-up strategy

The start-up strategy will be: Buying a domain name from a company with its corresponding IP address for the server. Hosting the system. Finally, available internet connection for users.

5.6 Conclusion and recommendation

5.6.1 Conclusion

This project has successfully delivered the StudentPortal system, a comprehensive web-based academic management solution designed to address the significant challenges faced by higher education institutions in Ethiopia. The system was developed as part of the internship program at RRT Company, demonstrating the company's commitment to innovation and talent development.

The StudentPortal system effectively addresses the problems of the existing manual system by providing a digital solution that automates key academic processes, ensures data security, improves efficiency, and enhances transparency. The system's features, including user authentication with role-based access control, student registration and approval workflows, department and course management, attendance tracking, assignment management, result entry and CGPA calculation, fee management, and transcript generation, provide a complete solution for academic management.

The technical implementation leverages modern, industry-standard technologies including ASP.NET Core 8, Angular 21, and PostgreSQL 15, ensuring the system is robust, scalable, and maintainable. The object-oriented design methodology, combined with UML modeling, provided a solid foundation for the system's architecture and implementation.

The feasibility analysis confirmed that the system is operationally, technically, behaviorally, and economically feasible. The cost-benefit analysis shows a positive return on investment within the first year of operation, making the system a viable solution for Ethiopian higher education institutions.

Through this project, valuable experience was gained in full-stack web development, database design, API development, security implementation, and project management. The internship at RRT Company provided a professional environment for applying theoretical knowledge to real-world software development.

56.2 Recommendations

Based on the experience gained during the development of the StudentPortal system, the following recommendations are made:

- **Phased Deployment:** The system should be deployed in phases, starting with a pilot institution, to gather feedback and make improvements before a wider rollout.
- **Amharic Language Support:** Future versions should include Amharic language support to make the system accessible to a wider audience in Ethiopia.
- **Mobile Applications:** Native mobile applications for Android and iOS should be developed to improve accessibility for students who primarily use smartphones.
- **SMS and Email Notifications:** Integration with SMS and email services should be added to enable automatic notifications for important events such as grade releases, fee deadlines, and announcements.
- **Legacy System Integration:** APIs should be developed to enable integration with existing institutional systems such as HR and finance platforms.
- **Advanced Analytics:** Machine learning and data analytics capabilities should be added to provide predictive insights on student performance and retention.
- **Library and Hostel Modules:** Additional modules for library management and hostel management should be developed to expand the system's coverage.
- **User Training:** Comprehensive user training programs should be developed and implemented to ensure smooth adoption of the system.
- **Performance Monitoring:** A monitoring and alerting system should be implemented to proactively identify and address performance issues.
- **Community Building:** A community of practice should be established among institutions using the system to share best practices and provide mutual support.

6.2 References

- [1] Ambler, S. W. (2004). The Object Primer: Agile Model-Driven Development with UML 2.0. Cambridge University Press.
- [2] EdrawMax, "ERD and Flowchart Design Tool," Edrawsoft, 2025. [Online]. Available: <https://www.edrawsoft.com/>. [Accessed: Jan. 12, 2026].
- [3] Diagrams.net, "UML Diagramming and System Architecture Mapping," Draw.io, 2025. [Online]. Available: <https://app.diagrams.net/>. [Accessed: Jan. 12, 2026].
- [4] Nguyen, T. H., & Le, H. T. (2020). Design and implementation of a web-based student management system. Journal of Information Technology Education, 19, 215-234.
- [6] PostgreSQL Global Development Group. (2024). PostgreSQL 15 Documentation. <https://www.postgresql.org/docs/15/>
- [7] Visual Paradigm, "Online Productivity Suite," Visual Paradigm, 2026. [Online]. Available: <https://online.visual-paradigm.com/>. [Accessed: Jan. 13, 2026].
- [8] Quest Journals, "Paper Title (use style: paper title)," Journal of Software Engineering Studies, vol. 9, no. 4, pp. 53-61, 2025. [Online]. Available: <https://www.questjournals.org/jses/papers/Vol9-issue-4/09045361.pdf>. [Accessed: Jan. 13, 2026].

Appendix

Appendix A: Source Code Structure

The project follows a clean architecture pattern with the following directory structure:

StudentPortal/

```
|— Backend (ASP.NET Core 8)
| |— Controllers/
| | |— AuthController.cs
| | |— DepartmentController.cs
| | |— CourseController.cs
| | |— StudentController.cs
| | |— FacultyController.cs
| | |— AttendanceController.cs
| | |— AssignmentController.cs
| | |— ResultController.cs
| | |— FeeController.cs
| | |— TranscriptController.cs
| | |— NoticeController.cs
| |— Services/
| | |— AuthService.cs
| | |— DepartmentService.cs
| | |— CourseService.cs
| | |— AttendanceService.cs
| | |— AssignmentService.cs
| | |— ResultService.cs
| | |— FeeService.cs
| | |— TranscriptService.cs
| | |— NoticeService.cs
| |— Models/
| | |— User.cs
| | |— Student.cs
| | |— Faculty.cs
| | |— Department.cs
| | |— Course.cs
| | |— StudentCourse.cs
| | |— Attendance.cs
| | |— Assignment.cs
| | |— AssignmentSubmission.cs
```

```

| | | FeeRecord.cs
| | | Notice.cs
| | DTOs/
| | Data/
| | Program.cs
|
| — Frontend (Angular 21)
| | src/
| | | app/
| | | | components/
| | | | | auth/
| | | | | dashboard/
| | | | | attendance/
| | | | | assignments/
| | | | | results/
| | | | | fees/
| | | | | transcripts/
| | | | | notices/
| | | | | settings/
| | | | services/
| | | | guards/
| | | | interceptors/
| | | assets/
| | angular.json
|
| Database/
| | migrations/

```

Appendix B: API Documentation (Swagger)

The API documentation is auto-generated using Swagger/OpenAPI and is accessible at the following endpoint: <https://api.studentportal.com/swagger>

The API follows RESTful conventions and supports the following HTTP methods:

- POST /api/auth/login - User authentication
- POST /api/auth/register - User registration
- GET /api/departments - List all departments

- POST /api/departments - Create a new department
- PUT /api/departments/{id} - Update a department
- DELETE /api/departments/{id} - Delete a department
- GET /api/courses - List all courses
- POST /api/courses - Create a new course
- GET /api/students - List all students
- POST /api/attendance - Mark attendance
- GET /api/attendance/{courseId}/{studentId} - Get attendance records
- POST /api/assignments - Create an assignment
- POST /api/assignments/submit - Submit an assignment
- POST /api/results - Enter results
- PUT /api/results/{id}/approve - Approve results
- GET /api/fees/{studentId} - Get fee records
- POST /api/fees - Record a fee payment
- GET /api/transcripts/{studentId}/generate - Generate transcript PDF
- GET /api/notices - List notices
- POST /api/notices - Create a notice

Appendix C: Database Schema

The complete database schema is defined in the Entity Framework Core migrations. The schema follows the design described in Chapter 4 (Section 4.7). The database includes 8 primary tables with appropriate foreign key constraints, indexes, and check constraints.

Key database features include:

- Auto-incrementing primary keys (SERIAL type)
- Foreign key constraints with ON DELETE CASCADE/RESTRICT
- Unique constraints on Email, Department Name, and Course Code
- Check constraints on scores (0-100 range) and CGPA (0-4.0 range)
- Composite indexes for frequently queried columns

- Timestamp fields with timezone support (TIMESTAMPTZ)
- Decimal precision for financial amounts (18,2) and grades (5,2)

Appendix D: Testing Plan

The StudentPortal system was tested using a comprehensive testing strategy that includes unit testing, integration testing, and user acceptance testing.

Table D.1: Testing Plan Overview

Test Type	Scope	Tools	Status
Unit Testing	Individual services and components	xUnit (Backend), Jasmine (Frontend)	Completed
Integration Testing	API endpoints and database operations	Postman, Swagger	Completed
Functional Testing	Complete user workflows	Manual testing	Completed
Performance Testing	Load and stress testing	Apache JMeter	Completed
Security Testing	Authentication, authorization, input validation	OWASP ZAP	Completed
User Acceptance Testing	End-user validation	Beta users (Students, Faculty)	Completed

Table D.2: Key Test Cases

Test ID	Test Case	Expected Result	Status
TC-001	Login with valid credentials	Successful login and redirect to dashboard	Pass
TC-002	Login with invalid credentials	Error message displayed	Pass
TC-003	Login with unapproved account	Pending approval message displayed	Pass

TC-004	Submit assignment with >= 75% attendance	Assignment submitted successfully	Pass
TC-005	Submit assignment with < 75% attendance	Submission blocked with error message	Pass
TC-006	Submit assignment with non-PDF file	File type validation error	Pass
TC-007	Enter and calculate grades	Total score and grade auto-calculated correctly	Pass
TC-008	Approve results by admin	Results become visible to student	Pass
TC-009	Generate transcript PDF	PDF generated with correct data	Pass
TC-010	Record fee payment	Payment recorded with correct amount and status	Pass